

Práctica 2

wu_javvy

Octubre 2025

1. Elegid una población determinada (puede ser de habitantes de una localidad, ciudad, país, o una determinada población animal localizada o no) y buscad datos sobre esa población en una serie de diez años consecutivos.

Resultado 1.1. *En mi caso, he tomado la población de un país de Europa del Este, Moldavia 🇲🇩, puesto que es una zona en la que en los últimos años se está experimentando una regresión demográfica, va a ser interesante ver el alcance y potencial que puede llegar a tomar la situación.*

Si tomamos los últimos diez años, dada la evolución demográfica vivida en Moldavia tras la caída de la URSS, vemos que el modelo se puede hacer más liviano, puesto que podemos obviar fenómenos extremos basados en política o guerras próximas al ser en gran parte el decrecimiento durante estos años por causas naturales :

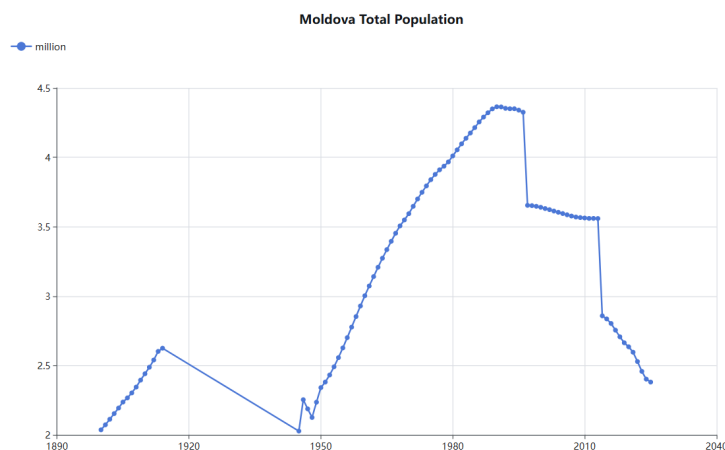


Figura 1: Población de Moldavia desde ppios. de s. XX y anexión de la URSS

Year	Population Moldova	Population Transnistria (after 2014)
2005	3,595,000	-
2006	3,586,000	-
2007	3,577,000	-
2008	3,570,000	-
2009	3,566,000	-
2010	3,563,000	-
2011	3,560,000	-
2012	3,560,000	-
2013	3,559,000	-
2014	2,857,815	500,700
2015	2,835,978	474,500
2016	2,803,186	470,600
2017	2,755,189	469,000
2018	2,707,203	465,100
2019	2,664,224	465,200
2020	2,635,130	465,800
2021	2,595,809	465,300
2022	2,528,654	459,800
2023	2,457,783	455,700
2024	2,402,306	451,644

Cadro 1: Study of Moldavian population since 2005

Entonces estudiando estos últimos 10 años, tenemos que tomar en cuenta para nuestro modelo la natalidad (ratio de nacimientos y tasa de fertilidad), mortalidad (esperanza de vida y ratio de fallecimientos) y migración (en especial la emigración, podemos obviar efectos de la guerra de Ucrania debido a evitar la primera en 2014 y por una suave neutralización demográfica entre la llegada de ucranianos y la emigración de moldavos en 2022).

Dichos datos los daré en el siguiente apartado donde exploramos la construcción de una función logística.

2. Ajústense los datos anteriores a una función logística, es decir, encuéntrense los parámetros de la función que mejor aproxima (aproximación óptima en norma euclídea) a los datos obtenidos. Explíquese el procedimiento utilizado.

Year	Crude birth rate (per 1000)	Crude death rate (per 1000)	Crude migration rate (per 1000)
2014	14.3	13.8	-197.5
2015	14.4	14.1	-8.0
2016	14.3	13.7	-12.1
2017	13.2	13.3	-16.9
2018	12.8	13.8	-16.4
2019	12.2	13.7	-14.4
2020	11.7	15.5	-7.1
2021	11.3	17.5	-8.7
2022	10.7	14.3	-18.3
2023	9.8	13.8	-13.1
2024	9.8	14.0	-32.3

Cadro 2: Vital rates in Moldova under central government control (2014–2024)

Resultado 2.1. *Entonces nos hemos propuesto crear una función logística siguiendo la construcción hecha en el modelo de crecimiento sugerido en el pdf :*

- *Tasa de natalidad: $a = a_1 - c_1y(t)$*
- *Tasa de mortalidad: $b = b_1 + d_1y(t)$*
- *Y un nuevo elemento, tasa de migración: $e = e_1 + f_1y(t)$*

Con el nuevo elemento lo que intentaremos hacer es reflejar más aún la importancia del crecimiento natural, puesto que la tasa de emigración aumentará cuanto más "muerto" esté el país, así que a, b deben influir en e . Recordamos que a_1, b_1, e_1 son valores iniciales, luego a, b, e son "funciones" con variable t .

No obstante, primero vamos a construir la función y luego ya damos valores a todos los elementos:

1. *Calculamos el parámetro por el que varía la función : $\lambda = a - b + e = (a_1 - c_1y(t)) - (b_1 + d_1y(t)) + (e_1 + (a_1 - b_1)f_1y(t)) = (a_1 - b_1 + e_1) - (c_1 + d_1 - f_1)y(t) = \alpha - \beta y(t) \rightarrow \lambda = \alpha - \beta y(t)$.*

2. Calculamos cómo varía la función : $y'(t) = \lambda y(t) = (\alpha - \beta y(t))y(t) = \alpha y(t) - \beta y(t)^2 \rightarrow y'(t) = \alpha y(t) - \beta y(t)^2$.

3. Resolvemos la EDO y nos queda de función : $y(t) = \frac{\alpha/\beta}{1 + Ce^{-\alpha(t-t_0)}}$, $C = \frac{\alpha/\beta - y_0}{y_0}$.

Ahora sí, vamos a empezar a dar valores a los parámetros. Para ello, como se nos pide encontrar los parámetros de la función que mejor aproximen a los datos obtenidos, vamos a recurrir a MatLAB. Primero vamos a definir los parámetros a calcular respecto de datos conocidos:

- $a = a_1 + c_1 y(t) \rightarrow c_1 = \frac{a_1 - a}{y(t)}$.
- $b = b_1 + d_1 y(t) \rightarrow d_1 = \frac{b - b_1}{y(t)}$.
- $e = e_1 + f_1 y(t) \rightarrow f_1 = \frac{e - e_1}{y(t)}$.

Código 2.1. *function* [c,d,f]=aproxparatiempo

```

1 function [sc,sd,sf]=aproxparatiempo
2
3 syms c d f fc fd ff
4
5 A=[14.3, 14.4, 14.3, 13.2, 12.8, 12.2, 11.7,11.3,10.7,9.8,9.8]';
6 B=[13.8,14.1,13.7,13.3,13.8,13.7,15.5,17.5,14.3,13.8,14]';
7 E=[-197.5,-8,-12.1,-16.9,-16.4,-14.4,-7.1,-8.7,-18.3,-13.1,-32.3]';
8 P=[2857815,2835978,2803186,2755189, 2707203, 2664224, 2635130, 2595809,...
9     2528654,2457783,2402306]';
10 A=A/1000; B=B/1000; E=E/1000;
11 a=A(1)*ones(10,1); b=B(1)*ones(10,1); e=E(2)*ones(10,1);
12 %En 2014 hay un outlier de migracion.
13
14 exc=-(A(2:11)-a)./P(1:10);
15 exd=(B(2:11)-b)./P(1:10);
16 exf=(E(2:11)-e)./P(1:10);
17
18 fc=0; fd=0; ff=0;
19 for i=1:10
20     fc=fc+(c-exc(i))^2;
21     fd=fd+(d-exd(i))^2;
22     ff=ff+(f-exf(i))^2;
23 end
24 X = linspace(0, 10^-6, 10^6);
25 Fc=@(x) subs(fc,c,x);
26 y=zeros(10^6,1);
27 for i=1:10^6

```

```

28     y(i)=Fc(X(i));
29 end
30 [¬,ayuda]=min(y);
31 sc=X(ayuda);
32
33 Fd=@(x) subs(fd,d,x);
34 y=zeros(10^6,1);
35 for i=1:10^6
36     y(i)=Fd(X(i));
37 end
38 [¬,ayuda]=min(y);
39 sd=X(ayuda);
40
41 Ff=@(x) subs(ff,f,x);
42 y=zeros(10^6,1);
43 for i=1:10^6
44     y(i)=Ff(X(i));
45 end
46 [¬,ayuda]=min(y);
47 sf=X(ayuda);

```

(NOTA: e_{2014} asumiremos que es igual que e_{2015} por ser outlier (anexión de Crimea) comparado con el resto de datos y en vez de usar `linspace` ($10^{-6}, 10^{-6}$), hemos usado debido a la gran carga numérica de operaciones el comando `linspace` ($10^{-8}, 10^{-4}$).)

Y ya por fin, podemos definir $y(t)$ y podemos evaluarla en MatLAB para ver qué resultados aporta:

Hemos obtenido resultados $c = 0$, $d = 2'0002 * 10^{-10}$, $f = 0$, luego si $a_1 = 14,3 * 10^{-3}$, $b_1 = 13,8 * 10^{-3}$, $e_1 = -8 * 10^{-3}$, eso implica que sustituyendo tenemos:

$$y(t) = \frac{\frac{-7,5*10^{-3}}{0+2*10^{-10}-0}}{1 + \left(\frac{\frac{-7,5*10^{-3}}{0+2*10^{-10}-0} - 2857815}{2857815} \right) * e^{7,5*10^{-3}(t-2014)}}$$

En forma más compacta tenemos:

$$y(t) = \frac{-3,75 * 10^7}{1 + \left(\frac{-3,75*10^7}{2857815} - 1 \right) * e^{7,5*10^{-3}(t-2014)}} = \frac{-3,75 * 10^7}{1 - 14,1219 * e^{7,5*10^{-3}(t-2014)}}$$

3. Estúdiense la posibilidad de encontrar un modelo alternativo y, en tal caso, si es posible, ajústense los datos.

Resultado 3.1. Otro posible modelo a utilizar podría haber sido el modelo logístico discreto al cual le hubiéramos implementado también el parámetro de migración. Esto como comenté antes es algo muy necesario si queremos estudiar la población de un país considerado por muchos el peor de Europa

(seguimos hablando de Moldavia).

Entonces para este modelo tomaremos el descrito en el pdf y lo transformaremos de la siguiente manera:

1. Definimos la EDO del modelo modificada para la migración: $y_{n+1} = y_n + \alpha y_n - \beta y_n^2 = (1 + \alpha - \beta y_n) y_n$.
2. Definimos un nuevo elemento x_n , $x_n = \frac{\beta}{1 + \alpha} y_n$, y entonces redefinimos la EDO: $x_{n+1} = \frac{\beta}{1 + \alpha} y_{n+1} = \beta(1 - \frac{\beta}{1 + \alpha} y_n) y_n = (1 + \alpha)(1 - x_n) x_n$.
3. Definimos una ecuación en diferencias llamando $K = 1 + \alpha$ y que será la que usaremos de forma iterativa al no contar con solución cerrada : $x_{n+1} = K x_n (1 - x_n)$.

Esto último sabemos lo que significa, que hay que hacer un nuevo código en MatLAB para calcular las poblaciones esperadas, pero para ello tenemos que tener en cuenta una cosa, que en este caso no se habla si los parámetros son variables o fijos, luego para solucionar ya aproximar la solución mejor vamos a convertir a estas variables en funciones recursivas:

- $\alpha = a - b + e$, donde a : tasa natalidad, b : tasa mortalidad, e : tasa migración.
- $\beta = \frac{-\alpha}{K}$, donde K : raíz no nula de $\Delta y_n = a y_n + b y_n^2$ y que corresponde a la capacidad de carga, esta la tenemos de 2 al ser lo mismo que $-\frac{\alpha}{\beta} = 3,75 * 10^7$.

Y tenemos que depende de las siguientes tasas que varían en cada iteración:

- $a = a_1 - c_1 y(t) \rightarrow a_n = a_{n-1} - c y_{n-1}$.
- $b = b_1 - d_1 y(t) \rightarrow b_n = b_{n-1} - d y_{n-1}$.
- $e = e_1 + (a_1 - b_1) f_1 y(t) \rightarrow e_n = e_{n-1} + (a_{n-1} - b_{n-1}) f y_{n-1}$

(NOTA: en este caso sí que vamos a poder aplicar lo dicho en el apartado previo de manera correcta sobre e .)

Los resultados de los coeficientes c, d, f los obtendremos de manera parecida al primer modelo con el siguiente código que simula qué valores deberían de tener teóricamente al despejar en las definiciones de a_n, b_n, e_n :

Código 3.1. `function [c,d,f]=aproxpararecursividad`

```

1 function [sc,sd,sf]=aproxpararecursividad
2
3 syms c d f fc fd ff
4
5 A=[14.3, 14.4, 14.3, 13.2, 12.8, 12.2, 11.7,11.3,10.7,9.8,9.8]';
6 B=[13.8,14.1,13.7,13.3,13.8,13.7,15.5,17.5,14.3,13.8,14]';
7 E=[-197.5,-8,-12.1,-16.9,-16.4,-14.4,-7.1,-8.7,-18.3,-13.1,-32.3]';
8 P=[2857815,2835978,2803186,2755189, 2707203, 2664224, 2635130, 2595809,...
9     2528654,2457783,2402306]';
10 A=A/1000; B=B/1000; E=E/1000;
11
12 exc=(A(2:11)-A(1:10))./P(1:10);
13 exd=(B(2:11)-B(1:10))./P(1:10);
14 exf=(E(2:11)-E(1:10))./(A(1:10)-B(1:10)).*P(1:10));
15
16 fc=0; fd=0; ff=0;
17 for i=1:10
18     fc=fc+(c-exc(i))^2;
19     fd=fd+(d-exd(i))^2;
20     ff=ff+(f-exf(i))^2;
21 end
22 X = linspace(0, 10^(-6), 10^6);
23 Fc=@(x) subs(fc,c,x);
24 y=zeros(10^6,1);
25 for i=1:10^6
26     y(i)=Fc(X(i));
27 end
28 [¬,ayuda]=min(y);
29 sc=X(ayuda);
30
31 Fd=@(x) subs(fd,d,x);
32 y=zeros(10^6,1);
33 for i=1:10^6
34     y(i)=Fd(X(i));
35 end
36 [¬,ayuda]=min(y);
37 sd=X(ayuda);
38
39 Ff=@(x) subs(ff,f,x);
40 y=zeros(10^6,1);
41 for i=1:10^6
42     y(i)=Ff(X(i));
43 end
44 [¬,ayuda]=min(y);
45 sf=X(ayuda);

```

Y tenemos curiosamente que podemos ahorrarnos a,b,e como funciones recursivas al ser c=d=f=0, entonces x_n queda definido como :

$$x_{2014+n} = \frac{\frac{7,5 \cdot 10^{-3}}{3,75 \cdot 10^7}}{1 - 7,5 \cdot 10^{-3}} y_{2014+n} = \frac{2 \cdot 10^{-10}}{1 + 7,5 \cdot 10^{-3}} y_{2014+n}$$

Y tendremos que la función recursiva se queda como:

$$x_{2014+(n+1)} = (0,9925)x_{2014+n}(1 - x_{2014+n})$$

Y escribiremos el código de la siguiente manera (por complejidad en el tema de operar dada la gran cantidad que tenemos de población, complejidad al incluir una nueva variable x sobre la cual no hemos definido e_{2014+n} , vamos a intentar hacerlo sobre la función logística inicial):

Código 3.2. *function* r=recursividad(n)

```
1 function r= recursividad(n)
2
3 a=14.3e-3; b=13.8e-3; e=-8e-3; K=3.75*10^7;
4 r=zeros(n+1,0); r(1)=2857815;
5
6 if n~=1
7     for N=1:n
8         r(N+1) = r(N) + (a-b+e) * (r(N)) - (a-b+e) / K * (r(N))^2;
9     end
10 end
```

4. Analícnse los resultados obtenidos, razonando las posibles disparidades entre los modelos y los datos

Resultado 4.1. Para analizar el error, vamos a hacerlo como antes por mínimos cuadrados para ver cómo ha sido la potencia de sendos métodos y cuánto se han aproximado a la evolución real de la población moldava:

Year	Diff Moldova-MC	Diff Moldova-MLD
2014	-3	0
2015	1,128	-2,036
2016	-8,894	-15,154
2017	-34,316	-43,602
2018	-59,921	-72,164
2019	-80,711	-95,843
2020	-87,806	-105,760
2021	-105,315	-126,026
2022	-150,845	-174,249
2023	-200,276	-226,310
2024	-234,496	-263,097

Cadro 3: Study of population diff between reality and simulations

1. Squared difference Moldova-M.C.:

$$\frac{\sqrt{\sum_{n=2014}^{2024} df MC_n^2}}{11} = 3,497550e + 04$$

2. *Squared difference Moldova-M.L.D.:*

$$\frac{\sqrt{\sum_{n=2014}^{2024} df MLD_n^2}}{11} = 4,008216e + 04$$

Hay que hacer varias observaciones después de todo lo recogido y expuesto en este trabajo sobre estas aproximaciones por funciones logísticas:

1. *Aun decreciendo la tasa de natalidad moldava, hemos tenido en sendos casos que el coeficiente que relacionaría esta con la población total da 0, esto puede darse por intentar ligar dicha tasa con la población en vez de estimarla como una función propia respecto de t o bien asumir un nuevo parámetro con la que relacionarla como la tasa de vejez o de esperanza de vida para reflejar su bajada.*
2. *En lo que coincide la realidad y nuestros modelos es en la confirmación de regresión demográfica que sufre Moldavia, con una tasa de natalidad que baja gravemente dada la estabilidad de la tasa de mortalidad y por la cual siempre hemos intentado relacionar dicha diferencia entre tasas con la emigración, puesto que como comenté en un primer momento, en un país destinado a morir nadie quiere ser parte de su daño colateral.*
3. *En ambos modelos nos falta un mayor reflejo de dichos cambios demográficos en término de tasas, lo cual tal vez se podría solucionar de cierta manera con herramientas de aproximación numérica (como veremos a continuación).*
4. *Por tanto, como sí hemos considerado varianza de la tasa de mortalidad en el primer caso (la función logística cerrada), es fácil comparar y ver que el modelo MC ha sido "el más potente", pero ambos han sido muy conservadores a la hora de analizar la tasa de natalidad y la migración, lo cual ha hecho que las poblaciones decrezcan de manera mucho más suave que en la realidad. Entonces me he querido preguntar (puesto que la tasa de mortalidad en estos 10 años tiene estabilidad suficiente para no tomar en cuenta sus alteraciones en la línea temporal) qué pasaría si altero de manera artificial dichas tasas en ambas funciones:*

Para ello (aunque sean resultados de aproximación numérica y sin ellos, la otra opción dentro de nuestro campo podría ser tomar más variantes que influyan en la natalidad y migración) vamos a hacer lo siguiente:

a) *En el primer caso:*

- $c \sim c(t) = \text{polyfit}(A, Va, 2) * [t^2, t, 1]'$, donde A es el vector de años que tenemos para calcular (2015 a 2024), Va es el vector con los valores de c calculados al despejar asociados a esos años.
- $d \sim d(t) = \text{polyfit}(A, Vb, 2) * [t^2, t, 1]'$, donde A es el vector de años que tenemos para calcular (2015 a 2024), Va es el vector con los valores de d calculados al despejar asociados a esos años.
- $f \sim f(t) = \text{polyfit}(A, Ve, 2) * [t^2, t, 1]'$, donde A es el vector de años que tenemos (2015 a 2024), Ve es el vector con los valores de f calculados al despejar asociados a esos años.

Obteniendo así:

- $c \sim c(t) = \text{polyfit}(A, Va, 2) * [t^2; t; 1] = (-1,895536e - 12)t^2 + (7,4366480e - 09)t - 7,288455e - 06$.
- $d \sim d(t) = \text{polyfit}(A, Ve, 2) * [t^2; t; 1] = (-2,114990e - 11)t^2 + (8,5465542e - 08)t - 8,633991e - 05$.
- $f \sim f(t) = \text{polyfit}(A, Ve, 2) * [t^2; t; 1] = (-1,555141e - 10)t^2 + (6,2760095e - 07)t - 6,331957e - 04$.

Y nos queda:

$$y(t) = \frac{\frac{-7,5*10^{-3}}{c(t)+d(t)-f(t)}}{1 + \left(\frac{\frac{-7,5*10^{-3}}{c(t)+d(t)-f(t)} - 2857815}{2857815}\right) * e^{7,5*10^{-3}(t-2014)}}$$

- b) Y en el segundo caso, vamos a hacer también cambios sobre las c, d, f teóricas quedando el nuevo código así (ya vamos a calcular dentro del programa los polinomios para hacer más liviana la memoria, en este caso se usará interpolación de Lagrange):

Código 4.1. `function r=N_recurividad(n)`

```

1 function r= N_recurividad(n)
2     A = [14.3, 14.4, 14.3, 13.2, 12.8, 12.2, ...
3         11.7, 11.3, 10.7, 9.8, 9.8] ' / 1000;
4     B = [13.8, 14.1, 13.7, 13.3, 13.8, 13.7, 15.5, 17.5, ...
5         14.3, 13.8, 14] ' / 1000;
6     E = [-8, -8, -12.1, -16.9, -16.4, -14.4, -7.1, -8.7, -18.3, ...
7         -13.1, -32.3] ' / 1000;
8     P = [2857815, 2835978, 2803186, 2755189, 2707203, 2664224, ...
9         2635130, 2595809, 2528654, 2457783, 2402306]';
10    Y = (2015:2024)';
11
12    Va = -(diff(A) ./ P(1:end-1));
13    Vb = (diff(B) ./ P(1:end-1));

```

```

14     Ve = (diff(E) ./ ((A(1:end-1)-B(1:end-1)).*P(1:end-1)));
15
16     pa = lagrangme(Va, Y(1:end), t);
17     pb = lagrangme(Vb, Y(1:end), t);
18     pe = lagrangme(Ve, Y(1:end), t);
19
20     K = -7.5e-3 / (Va(1) + Vb(1) - Ve(1));
21
22     a = 14.3e-3; b = 13.8e-3; e = -8e-3;
23     ayA = zeros(n+1,1); ayA(1) = a;
24     ayB = zeros(n+1,1); ayB(1) = b;
25     ayE = zeros(n+1,1); ayE(1) = e;
26     r = zeros(n+1,1); r(1) = 2857815;
27
28     for N = 1:n
29         year = 2014 + N;
30         c_val = subs(pa, t, year);
31         d_val = subs(pb, t, year);
32         f_val = subs(pe, t, year);
33
34         growth = (ayA(N) - ayB(N) + ayE(N));
35         r(N+1) = r(N) + growth*r(N) - (growth/K)*(r(N)^2);
36
37         ayA(N+1) = ayA(N) - c_val*r(N);
38         ayB(N+1) = ayB(N) + d_val*r(N);
39         ayE(N+1) = ayE(N) + f_val*(ayA(N)-ayB(N))*r(N);
40     end
41 end

```

(NOTA: Este método sólo nos sirve para aproximar con datos conocidos, porque por la variación que hay en ciertos términos, el modelo logístico diverge, pero para lo que se nos pide que es aproximarlos a los datos demográficos de los últimos 10 años nos sirve.)

Código 4.2. `function sol=lagrangme(F,X,y)`

```

1 function sol=lagrangme(F,X,y)
2
3 if nargin==2
4     syms y
5 end
6 syms x lagrange ayuda
7 lagrange = 0;
8 for i=1:length(X)
9     ayuda=1;
10    for j=1:length(X)
11        if j≠i
12            ayuda=ayuda*(x-X(j))/(X(i)-X(j));
13        end
14    end
15    lagrange = lagrange + ayuda*F(i);
16 end
17 if strcmp(class(y), 'sym')

```

```

18     sol=subs(lagrange,x,y);
19 else
20     sol=zeros(length(y),1);
21     for j=1:length(y)
22         a=y(j);
23         sol(j)=subs(lagrange,x,a);
24     end
25 end
26 end

```

Year	Diff Moldova-NMC	Diff Moldova-NMLD
2014	0	0
2015	14,013	-168
2016	6,037	-10,210
2017	-22,382	-25,005
2018	-49,726	-24,445
2019	-64,992	-18,653
2020	-53,761	-4,090
2021	-35,468	-14,299
2022	-24,424	-41,924
2023	4,901	-55,207
2024	70,853	-67,044

Cadro 4: New study of population diff between reality and simulations

a) *Squared difference Moldova-N.M.C.:*

$$\frac{\sqrt{\sum_{n=2014}^{2024} dfNMC_n^2}}{11} = 1,192931e + 04$$

b) *Squared difference Moldova-N.M.L.D.:*

$$\frac{\sqrt{\sum_{n=2014}^{2024} dfNMLD_n^2}}{11} = 9,619426e + 03$$

5. *Por último, sería curioso ver qué futuro le tenemos deparado al nuestra querida nación moldava bajo el modelo más potente visto (y que pueda ser aplicado con los datos que tenemos) (NMC):*

<i>Year</i>	<i>Population Moldova</i>	<i>Change</i>
<i>2024</i>	<i>2,331,453</i>	<i>-121,429</i>
<i>2025</i>	<i>2,191,673</i>	<i>-139,780</i>
<i>2026</i>	<i>2,038,111</i>	<i>-153,562</i>
<i>2027</i>	<i>1,876,293</i>	<i>-161,818</i>
<i>2028</i>	<i>1,711,900</i>	<i>-164,393</i>
<i>2029</i>	<i>1,550,067</i>	<i>-161,833</i>
<i>2030</i>	<i>1,394,927</i>	<i>-155,140</i>
<i>2031</i>	<i>1,249,429</i>	<i>-145,498</i>
<i>2032</i>	<i>1,115,369</i>	<i>-134,060</i>
<i>2033</i>	<i>993,565</i>	<i>-121,804</i>
<i>2034</i>	<i>884,082</i>	<i>-109,483</i>

Uno muy triste, aunque tomemos en cuenta que nuestro modelo espera un descenso demográfico de la población actual moldava más extremo que el que podríamos esperar. Tal vez una moderación de la importancia de la tasa de migración o que no dependa en sí de la población, si no más del crecimiento natural son posibilidades que pueden hacer más creíble esta simulación.